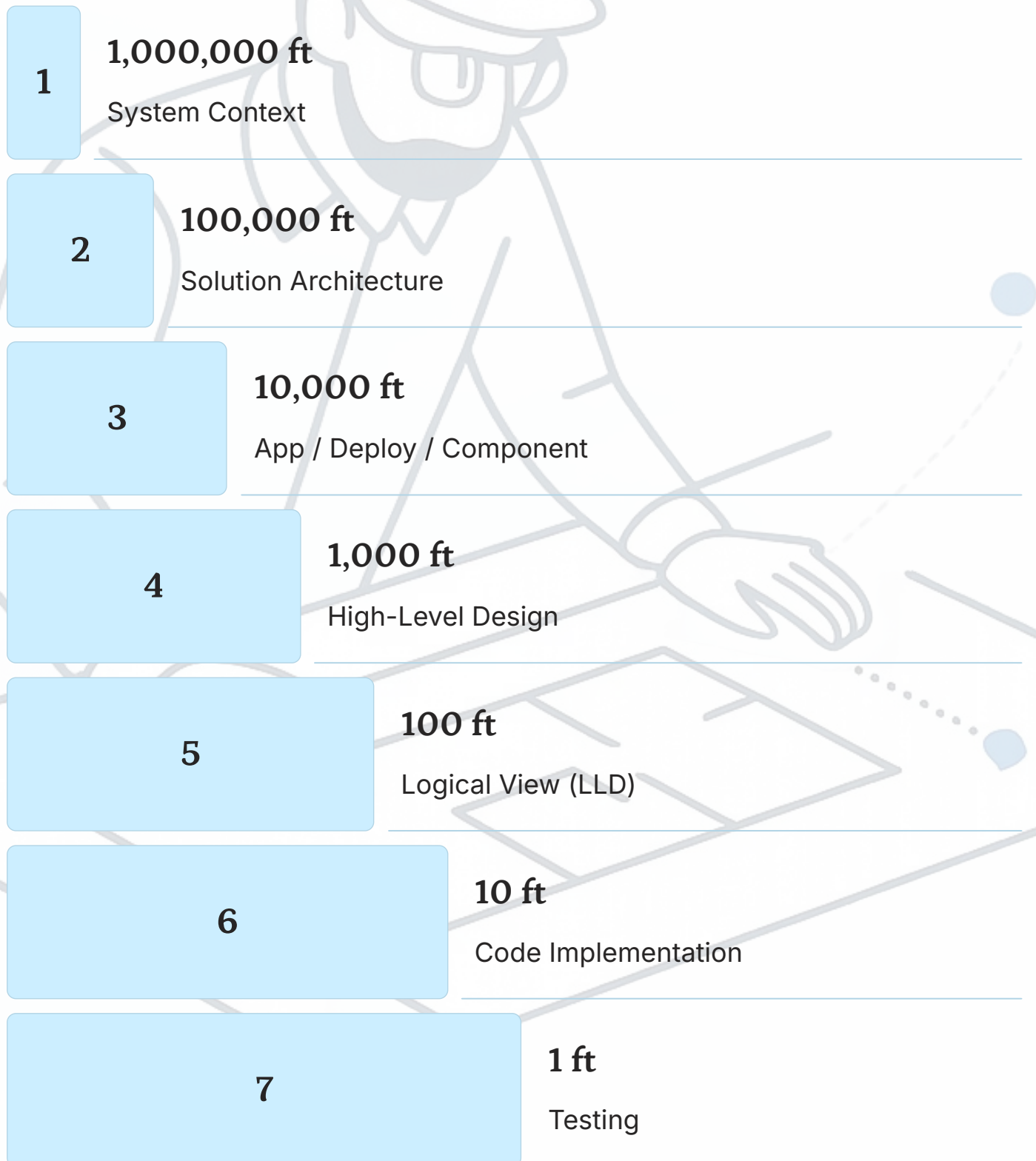


The Seven-Step Architecture & Build Framework

The framework you will use for **every** non-trivial system you design — moving from **abstract** → **concrete (top-down)**.



Why This Framework Exists

Most engineers go straight from a vague brief to writing code. They miss two critical things that cause systems to fail in production.

Problem 1

Implicit assumptions stay implicit.

The codebase ships, the assumptions break in production. What seemed obvious in your head was never written down — and never challenged.

Problem 2

NFRs get bolted on later.

Performance, scale, reliability, security — by the time you remember them, the architecture cannot accommodate them cheaply. Retrofitting is always more expensive than designing for them upfront.

The seven steps force you to make those assumptions explicit and to design *for* the NFRs, not around them.

- ① The other rule: **layers and dimensions, not just code**. Code is the smallest, last layer. If your design conversation starts at "which class do I write," you've already lost altitude.

Step 1 — System Context

(1,000,000 ft)

WHERE IS THIS SYSTEM IN THE WORLD? WHO USES IT? WHY DOES IT EXIST?

What to Produce

- A **1-page description** of the system you are building
- A **justification** for why this system needs explicit architecture (or doesn't)
- **Statement of scale, complexity, and criticality**
- **Comparison anchors** — point to real-world systems with similar shape (YouTube, Facebook, Uber, LinkedIn) and say what's like them and what's different
- **High-level components** and external ecosystem integrations (auth providers, payment gateways, third-party data sources)

How to Think

Start *outside* the system. Ask yourself:

- What does the user do **five seconds before** the system gets involved?
- What does the system do **five seconds after** the user interacts?
- Who are all the actors — human and machine — that touch this system?
- What would break in the world if this system went down?

□ This is the altitude where you define **why the system exists** — not how it works.

Step 2 — Solution Architecture (100,000 ft)

THINK LIKE A CTO. PICK THE TECHNOLOGIES. JUSTIFY EACH ONE.

At this altitude you are deciding the *menu*, not cooking. The next two steps cook. Focus on **what technologies and why**, not deep implementation.

Layer	Decision Area	Trade-off Considered	Constraint Satisfied
Backend	Runtime / framework choice	Monolith vs microservices	Team size, deployment cadence
Frontend	SPA vs SSR vs static	SEO vs interactivity	Accessibility, performance
Database	SQL vs NoSQL	Consistency vs flexibility	Data shape, query patterns
Cache	Cache vs search engine	Read latency vs query power	p99 latency target
Messaging	Sync vs streaming	Simplicity vs throughput	Reliability, decoupling
API Style	REST vs GraphQL vs gRPC	Flexibility vs performance	Client diversity
AI / ML	Hosted model vs self-hosted	Cost vs control	Data privacy, latency

 Each row in your choice table must name the **constraint** the choice satisfies — not just the technology name.

Step 3 — App / Deploy / Component Architecture (10,000 ft)

BREAK THE SYSTEM INTO MAJOR COMPONENTS. SHOW HOW THEY FIT TOGETHER.

Zoom in until you see boxes that map 1:1 to repositories or services. Not lines of code — boxes. Produce **six distinct views** of the same system.



Component View

Major services(auth, user, search, billing)and how they call each other



Application View

Layers (frontend / backend / data / integration), tiers, and the technology element in each layer



Deployment View

What runs where — pod, region, CDN edge — how scaling happens, what's behind which load balancer



Security View

Authentication flow, authorisation boundaries, trust zones, secrets management, encryption boundaries



Compliance View

Data governance, audit logging.



Data View

Data flow across component



Also note: scaling challenges, data flow direction, and external integrations at this altitude.

Step 4 — High-Level Design / HLD

(1,000 ft)

KPI / NFR DRIVEN — THIS IS WHERE MOST SYSTEM DESIGN EXERCISES SECRETLY FAIL.

For each KPI listed below, produce two statements: **how it will be achieved** (mechanism) and **how it will be measured** (metric, threshold, alarm).

1



Security

Authn/authz, secret handling, data-at-rest / data-in-transit encryption boundaries

2



Performance

Latency p50 / p95 / p99, throughput RPS — measured, not assumed

3



Scalability

Horizontal headroom, what scales first, what bottlenecks first

4



Reliability

Availability SLO, blast radius of each component, retry / circuit-breaker policy

5



Compliance

Data residency, audit trail, retention policies

6



Maintainability

Repository structure, testing strategy, operational runbooks

7



Accessibility

WCAG level, i18n, screen-reader behaviour

For every KPI, write a sentence that starts "**we will achieve this by ...**" and a sentence that starts "**we will know this is working because ...**"

Step 3 listed the boxes. Step 4 says **why each box looks the way it does** — and the "why" is always one of the KPIs above.

Step 5 — Low-Level Design / LLD

(100 ft)

ZOOM IN ON THE COMPONENTS FROM STEP 3. SHOW HOW EACH ONE IS BUILT.

What to Produce

1 Classes / Modules / APIs

Inside each component — the internal structure that makes the component work

2 Data Models

Tables, columns, indexes, foreign keys, document shapes, message schemas

3 Sequence Diagrams

Internal logic for the 2 most important flows in the system

4 KPI Mapping

Each design decision mapped back to the KPIs from Step 4

5 Code Structure

Directory layout, module boundaries, layering (controllers / services / repositories), naming conventions

How to Think

✓ If Step 4 is **what we promise**, Step 5 is **the receipt that proves we can deliver it**.

At this altitude, every design decision should be traceable back to a KPI. If you cannot explain why a class exists in terms of a non-functional requirement or a functional requirement, question whether it should exist at all.

The layering pattern — controllers / services / repositories — is not arbitrary. It enforces separation of concerns that makes each layer independently testable, which feeds directly into Step 7.

Step 6 — Code Implementation (10 ft)

PICK 1–2 CRITICAL USE CASES AND ACTUALLY BUILD THEM.

You are *not* required to build the full system. You are required to make the code align with the architecture. Pick the use cases that are most architecturally significant — the ones where getting it wrong would invalidate the most design decisions.

Working Code

For the chosen 1–2 use cases — functional, runnable, not pseudocode



Reflects Step 5

Proper structure, layering, named modules, documented APIs — the code must mirror the LLD



The Golden Rule

If your code disagrees with your LLD, one of them is wrong. Fix the wrong one before continuing



Rule: If your code disagrees with your LLD, one of them is wrong. Fix the wrong one before continuing. Never let code and design drift apart — that drift is technical debt accumulating in real time.

Step 7 — Testing (1 ft, Ground Reality)

THE SYSTEM IS ONLY "COMPLETE" WHEN ALL TESTS PASS — 100% GREEN, NO EXCUSES.

The Rule

If you can't test it, you don't understand it well enough yet.

Go back to Step 5.

100% green or it isn't done. There is no partial credit at ground level.

What to Produce

- **Unit tests** — every module's behaviour with isolated inputs
- **Integration tests** — endpoints, DB operations, cross-component flows
- **High coverage** on critical paths — not just happy paths
- **Edge cases handled** — empty input, oversized input, malformed input, concurrent input

Key Principles — Apply at Every Step

Abstract → Concrete

Top-down. Always. No skipping levels because "the code is obvious." Every altitude has its own artefact and its own questions to answer.

Make Implicit Assumptions Explicit

Anything you assumed silently in your head goes on the page. Capacity, traffic shape, data freshness, user journeys — all of it.

Focus Heavily on NFRs (KPIs)

They are not "the boring part." They are the design. Functional requirements barely move architecture; NFRs *are* architecture.

Think in Layers and Dimensions

Component, deployment, data, security, compliance — these are *separate views* of the same system. Draw them separately.

Validate Through Testing, Not Assumptions

A claim is only true if a test proves it. "*We assume p99 < 200 ms*" is not a design statement. "*We measured p99 = 180 ms with 1K RPS*" is.

Artefact Reference Table

WHAT YOU SUBMIT AT EACH ALTITUDE

Every step has a concrete deliverable. If you cannot produce the artefact, you have not completed the step. Use this table as your checklist before moving to the next altitude.

Step	Altitude	Artefact You Submit
1	1,000,000 ft	System context page + comparison anchors (real-world systems with similar shape)
2	100,000 ft	Tech-choice table with rationale — one constraint per row
3	10,000 ft	Component diagram + deployment diagram + layers diagram (six views)
4	1,000 ft	KPI-by-KPI HLD table (Achieved by / Measured by) — all seven KPIs
5	100 ft	Class / module / API list + data model + sequence diagrams
6	10 ft	Working code for 1–2 critical use cases, aligned with LLD
7	1 ft	Unit + integration tests, 100% green — no exceptions

✓ Read **Step Guidelines.md** next. It is the **how-to companion** to this framework — the tactical detail behind each step above.